

DND Dice Reader



Introduction

The ideal goal of this project is to develop a system that can read off the die values for multiple types of dice used for dungeons and dragons (DND). The system should ideally take in an image of multiple dice and return the values of the different face-up values. This system should also ideally be able to work on a wide set of dice including ones not included in the training set.

Motivation

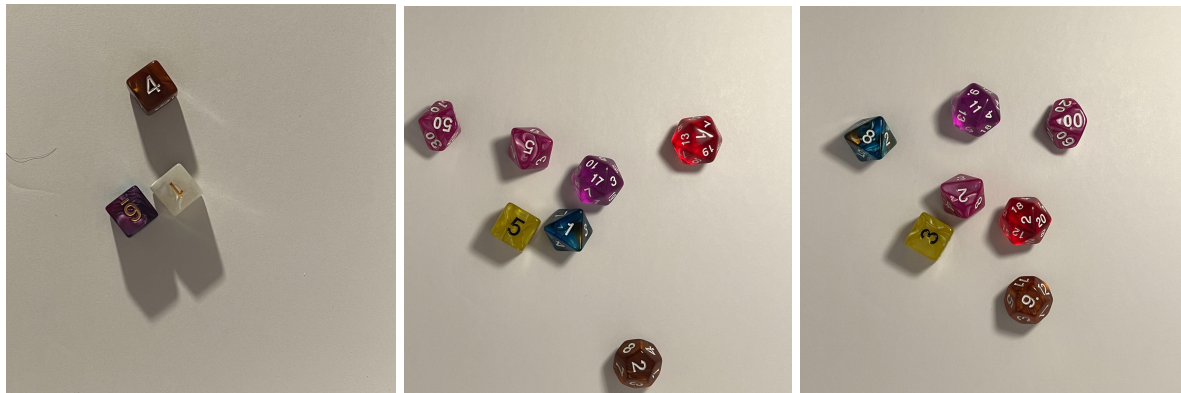
DND is a popular tabletop roleplaying game. Much of the game involves rolling dice to determine the outcome of various actions. While fun, this can get cumbersome in cases where a player has to roll multiple dice to determine the effect of a single action. Doing this multiple times in a session can cause it to slow down as players sum small numbers together over and over again. A system which automatically reads the dice and performs basic arithmetic for players would make the game flow smoother.

Another benefit of the system is for blind or low-sight players who can start enjoying the feeling of owning and rolling their own dice instead of relying on a flat random number generator on their devices.

Dataset Information

To create the data used in the model, 15 unique DND dice sets. Each dice sets contain 7 different die types, these dice are classified based on how many sides it has. A regular 6 sided die is called a d6. In dnd, we have the d4, d6, d8, d10 (single digit), d10 (percentile die), d12, and the d20. The percentile die is a version of the d10 with a 0 added at the end, the two d10 versions are typically used together to simulate a d100.

To create the dataset, all 105 of these dice were collected into a bag. Handfuls of these dice were then taken and rolled onto a white background. A picture of these dice were then taken with a distant camera that was zoomed. The idea behind this is that generating additional training data is easier during data augmentation down the line if images were taken in such a way that perspective effects were consistent. Specifically, we want the images to be taken from the perspective of an infinite-distant looking perpendicularly down onto the surface. Some examples of these images are shown below.



These images were then labeled with the bounding boxes of each die and then split into subimages based on the bounding boxes. These images of individual die were then labeled. Overall, the data set contains 275 total images. In these 275 images, we have over 1241 individual die faces, with images of d6 faces composing a large majority (~33%) of all die face images.

Model Design

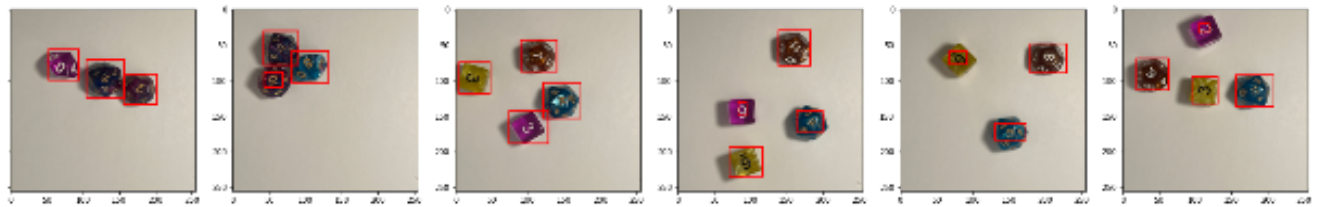
In order to function, our model will need to have two components. The first component needs to extract the dice in each image. In other words, it needs to identify the bounding boxes for all the dice. The second component will then take in these images of individual dice and classify them.

For the first component, we used a custom implementation of YoloNetV3 from a [Kaggle Notebook](#) originally used for counting the heads of wheat in an image. This YoloNetv3 implementation is different from most implementations since it only cares about extracting the locations of a specific object, and this object may be scattered all over the image. Because of the domain similarities, this was a natural base model to start from. A few modifications were made to this model, the first is the number of filters across all the CNN layers were reduced by a factor of four, shrinking the model. Another modification made was the addition of more aggressive image augmentation techniques such as greater allowed degrees for rotation. The final model takes in a given image as an input and returns the coordinates of likely bounding boxes within the image.

The second component which was used to classify the die faces takes in a die face and returns one of 28 labels (0, 1, 2, 3, ..., 80, 90). These labels are based off of all the possible die face values in DND. We note the type of die is not included in the classification here, so a 3 on a d4 and a 3 on a d8 would both be classified as a 3. To create this model, a pretrained version of mobilenet-v2 was used with minimal modifications. We finetuned all the weights of this model on the dataset including aggressive data augmentation. Using mobilenet-v2 as a baseline model is ideal since it is able to provide good out-of-the-box finetuning performance on many image classification tasks. In our case, the mobilenet-v2 model we used was pretrained on imagenet.

Evaluation

Both of the models used were initially trained on google colab using tensorflow. The image segmentation model was trained for 200 epochs on ~250 images. This model's losses are based on how well it can predict the size and the location of bounding boxes. The performance of the model on the validation set of 20 images is near-flawless, it identifies all the dice and places a box on them. However, sometimes these boxes are too small, as can be seen in the last three images below.



The dice face classification model performed similarly well. After finetuning the entirety of Mobilenet-V2 on 1097 images for 191 epochs, the training accuracy was 95.7% and the validation accuracy on a smaller set of 53 images was 92.5%.

We note that in the final version of the model deployed, the models are in float-32. It's theoretically possible to make the models smaller using int-8 quantization and model pruning. Additionally, we can perform model distillation by generating more (unlabelled) dice images and using the output of a stronger model to teach a smaller model. However due to time constraints, this process was not explored.

Deployment

To deploy the model, we decided to use TensorflowJS to make the model available as a local webapp. This process involved converting all the pre-processing and post-processing functions from python to javascript as well as converting the tensorflow models themselves into the tensorflowjs format. After all the conversions, the combined size of both models was around

~100Mb with the image segmentation model being around 25Mb and the mobilenet model being around 75Mb.

The model is available to use (as of now) on jmlrudan.github.io. The deployed model was tested using the same phone used to capture the images. To test the model, we created a bag of 8 dice never seen before by the model and evaluated the model on it until 150 dice rolls were reached. Under this setting, the model got 122 out of 150 dice rolls correctly attaining an accuracy of **81.3%**. This is lower than the accuracy as measured in colab (92.5%). There are two main reasons for why this could be the case. The first is that the model performs slightly worse on dice it has not seen, the second is that the distribution of dice are different. In the original training and validation set, d6 die composed a larger fraction of all dice pictures. Since d6 are easier to classify, this might have inflated the accuracy relative to evaluating on a set of randomly selected dice.

Below is an image of what the deployed web app looks like. A user can upload an image of dice and then the app returns images of the segmented dice along with the values it detected.

TensorflowJS Dice Reader



This web app demonstrates the use of TensorflowJS to locally classify images of DND dice. Upon uploading an image, the image is passed to a YOLOv3

The model works best when given a **square** image on a **white** background, with a **top down** view

Note for iPhones: The model works best when you take a square picture using the camera app instead of using the "take photo" feature.

IMG_4974.jpg

Current Dice sum: 15




2


7

Future Work/Potential Modifications

There are two main ways the model can still be improved: Accuracy and latency. In terms of accuracy, the model performs really well on d6 die since they compose a large majority of the training dataset. Expanding the dataset to have more copies of the different die varieties would likely increase the model's accuracy on classifications on them. In terms of latency, the model still can have its sized reduced by an order of magnitude, with int-8 quantization, model pruning, more data, and teacher-training, it might be possible to use unlabeled dice data which is relatively easy to generate to train better models.

Conclusion

The project is overall a success, since we were able to achieve our primary goal of having a somewhat reliable system to identify and count die values. A demo of the model in action can be seen [here](#)